DOCUMENTO TÉCNICO

ProCer360 Frontend

Documentación del sistema de gestión de proveedores, auditores y recursos

FECHA
31 mar 2026

RAMA
main

STACK
React 19 · Vite · Tailwind

Tabla de contenidos

01 Visión general del sistema

02 Stack tecnológico

03 Arquitectura y organización de archivos

04 Conexión con el backend

05 Autenticación y roles

06 Routing y navegación

07 Layout y componentes de navegación

08 Componentes principales y cómo se usan

09 Páginas y flujos

10 Servicios (capa API)

11 Hooks personalizados

12 Gestión de estado

13 Detalles de implementación relevantes

14 Sistema de diseño

15 Limitaciones y deuda técnica conocida

16 Cómo correr el proyecto

17 Usuarios de prueba

1. Visión general del sistema

ProCer360 / ContratistaFront es un sistema de gestión de proveedores, auditores y recursos para empresas.

Los usuarios pueden tener distintos roles y la interfaz cambia completamente según el rol activo. Los proveedores gestionan sus propios datos y documentos; las empresas administran a sus proveedores; los auditores revisan y aprueban documentación; el admin tiene acceso total.

Entidades principales

Entidad	Descripción
Proveedor	Empresa contratista. Tiene empleados, vehículos, maquinaria y documentos.
Empresa	Cliente que contrata proveedores. Pertenece a un grupo.
Grupo	Agrupa empresas. Define qué documentación exige a sus proveedores.
Auditor	Revisa documentación. Puede ser Técnico o Legal.
Recurso	Elemento de un proveedor: Empleado, Vehículo o Maquinaria.
Documento	Archivo adjunto a un proveedor o recurso. Tiene estado y vencimiento.

2. Stack tecnológico

Categoría	Tecnología	Versión	Propósito
Framework UI	**React**	19.x	Librería de componentes
Build tool	**Vite**	5.x	Dev server y bundler
Routing	**React Router DOM**	7.x	Navegación SPA
HTTP	**Axios**	1.x	Llamadas al backend
UI Components	**PrimeReact**	10.x	Tablas, calendarios, toasts, DataTable
UI Components	**Flowbite React**	0.12	Componentes adicionales
Estilos	**Tailwind CSS**	3.x	Utility-first CSS
Formularios	**React Hook Form**	7.x	Formularios performantes
Gráficos	**Chart.js** + react-chartjs-2	4.x / 5.x	Gráficos de dashboard
Íconos	**React Icons** + **PrimeIcons**	5.x / 7.x	Íconos en componentes
Locación	**country-state-city**	3.x	Selector de país/estado/ciudad

Nota React 19: La app usa la versión más reciente de React con el compilador SWC (más rápido que Babel). No se usa `Suspense` de forma extensiva aún.

---

3. Arquitectura y organización de archivos

```
src/
├── api/                    # Configuración base de HTTP
│   └── axiosConfig.js      # Instancia Axios con baseURL: http://localhost:8080
│
├── context/                # Estado global (React Context)
│   ├── AuthContext.jsx     # Autenticación, roles, sesión, timeout
│   ├── NotificationContext.jsx  # Sistema de toast notifications
│   └── BreadcrumbContext.jsx    # Labels dinámicos para breadcrumbs por ruta
│
├── components/             # Componentes reutilizables
│   ├── layout/             # Estructura de la app
│   │   ├── MainLayout.jsx  # Wrapper con sidebar + navbar + <Outlet>
│   │   ├── Navbar.jsx      # Barra superior con breadcrumbs y selector de rol
│   │   └── Sidebar.jsx     # Menú lateral dinámico por rol
│   ├── resources/          # Componentes de recursos y documentos
│   │   ├── AdminSupplierFilterModal.jsx
│   │   ├── DocumentEntityTable.jsx
│   │   └── forms/          # Formularios por tipo de recurso
│   │       ├── EmployeeData.jsx
│   │       ├── VehicleData.jsx
│   │       ├── MachineryData.jsx
│   │       ├── DocumentsData.jsx
│   │       ├── AssignmentData.jsx
│   │       ├── SupplierStepGeneral.jsx
│   │       ├── SupplierStepGroupCompany.jsx
│   │       ├── SupplierStepLocation.jsx
│   │       ├── SupplierStepContacts.jsx
│   │       └── SupplierStepDocuments.jsx
│   ├── suppliers/
│   │   └── StepHeader.jsx
│   └── ui/                 # Componentes atómicos reutilizables (25+)
│       ├── AppTable.jsx
│       ├── Badges.jsx
│       ├── ConfirmationModal.jsx
│       ├── DocumentCard.jsx
│       ├── Dropdown.jsx
│       ├── Input.jsx
│       ├── Label.jsx
│       ├── LoadingOverlay.jsx
│       ├── MultiSelect.jsx
│       ├── PageHeader.jsx
│       ├── PrimaryButton.jsx
│       ├── Select.jsx
│       ├── StatCard.jsx
│       └── WizardSteps.jsx
│
├── pages/                  # Páginas (una por ruta)
│   ├── auth/               # LoginPage, RoleSelectionPage
│   ├── dashboard/          # DashboardHome
│   ├── suppliers/          # SuppliersList, SupplierDetail, NewSupplier...
│   ├── auditors/           # AuditorsList, TechnicalAudit, LegalAuditDashboard...
```

```
│   ├── companies/          # CompanyList, NewCompany, EditCompany, CompanySettings
│   ├── resources/          # ResourcesDashboard + employees/ vehicles/ machinery/
│   ├── users/              # UsersList, UserDetail
│   └── errors/             # NotFound, ServerError
│
├── services/               # Capa de acceso a API (una por dominio)
│   ├── supplierService.js
│   ├── elementService.js   # Recursos: empleados, vehículos, maquinaria
│   ├── fileService.js
│   ├── groupService.js
│   ├── auditorService.js
│   ├── companyService.js
│   ├── dashboardService.js
│   └── userService.js
│
├── hooks/                  # Custom hooks (lógica reutilizable)
│   ├── useSupplier.js
│   ├── useSupplierForm.js
│   ├── useEmployees.js
│   ├── useMachinery.js
│   ├── useVehicles.js
│   ├── useResourceStats.js
│   └── usePermissions.js
│
├── utils/                  # Funciones puras de utilidad
│   ├── authUtils.js
│   ├── fileUtils.js
│   └── formatUtils.js
│
├── data/                   # Constantes y datos mock para desarrollo
│   ├── documentConstants.js
│   ├── supplierConstants.js
│   └── mock*.js
│
├── App.jsx                 # Router principal
├── main.jsx               # Entry point
└── index.css              # Estilos globales + overrides PrimeReact
```

Patrón de capas

```
Página (pages/)
  └── usa Hooks (hooks/) para lógica compleja
        └── llama a Services (services/) para API
              └── usa axiosConfig.js (api/) como cliente HTTP
```

Los componentes en `components/ui/` son atómicos y agnósticos del dominio. Las páginas los componen junto con datos del backend.

4. Conexión con el backend

Configuración base

`src/api/axiosConfig.js`

```
import axios from 'axios';
const api = axios.create({ baseURL: 'http://localhost:8080' });
export default api;
```

> El token JWT se guarda en localStorage pero **no se adjunta automáticamente** en cada request. Si el backend empieza a requerirlo, se necesita agregar un interceptor en `axiosConfig.js`.

Mapa de endpoints por dominio

AUTENTICACIÓN

Método	Endpoint	Body / Params
POST	`/auth/login`	`{ userName, password }`
GET	`/auth/users/:id`	—
GET	`/auth/users`	—

PROVEEDORES

Método	Endpoint	Params
GET	`/supplier/all`	—
GET	`/supplier/authorized`	`userId, role, entityId`
GET	`/supplier/byCompany/:id`	—
GET	`/supplier/oneSupplier`	`?cuit=`
GET	`/supplier/oneSupplierForDocument`	`?cuit=`
POST	`/supplier/save`	body: objeto proveedor
PUT	`/supplier/update`	body: objeto proveedor
DELETE	`/supplier/:id`	—
GET	`/supplier/associations`	`?cuit=`
POST	`/supplier/associate`	body: `{ cuit, companies[] }`

RECURSOS (ELEMENTOS)

Método	Endpoint	Params
GET	`/elements/getBySupplier`	`?id=`
GET	`/elements/getBySupplierAndActive`	`?idSupplier=&idActive=`
GET	`/elements/getBySupplierAndActiveType`	`?idSupplier=&idActiveType=`
GET	`/elements/authorized`	`?idActiveType=&userId=&role=&entityId=`
GET	`/actives/getByType/:id`	—
GET	`/elements/get/:id`	—
POST	`/elements/save`	body: objeto elemento
POST	`/elements/saveFile/:id`	body: `{ file, fileName, mimeType }`

GRUPOS Y REQUERIMIENTOS

Método	Endpoint	Params
GET	`/group/all`	—
POST	`/group/save`	body: objeto grupo
GET	`/group_requirements/details`	`?idSupplier=&idGroup=&idActiveType=`
GET	`/group_requirements/specific`	`?idSupplier=&idGroup=&idActiveType=`
GET	`/group_requirements/getByElement`	`?idElement=`
GET	`/group_requirements/specificResource`	`?idSupplier=&idGroup=&idActive=&idElement=`

ARCHIVOS

Método	Endpoint	Notas
GET	`/files/:id`	Responde como `blob`
DELETE	`/files/:id`	—

DASHBOARD

Método	Endpoint	Params
GET	`/dashboard/stats`	`?userId=&role=&entityId=`

EMPRESAS

Método	Endpoint	Params
GET	`/company/getAll`	—
GET	`/company/byGroup/:id`	—
POST	`/company/save`	body: objeto empresa
PUT	`/company/updateStatus`	`?idCompany=&requiredTechnical=`

AUDITORES

Método	Endpoint	Notas
GET	`/auditors/getAll`	—
GET	`/auditors/getById/:id`	—
POST	`/auditors/save`	—
POST	`/auditors/:id/assignCompany/:companyId`	—
DELETE	`/auditors/:id/removeCompany/:companyId`	—
GET	`/audit/pending`	—
POST	`/audit/save`	body: `{ fileId, status, observation }`

Cómo se renderizan los datos

El flujo estándar de una página:

```
1. Componente monta (useEffect con [])
2. Llama al servicio correspondiente (async/await)
3. Setea estado local (useState) con los datos
4. Mientras carga → muestra LoadingOverlay o spinner
5. Al completar → React re-renderiza con los datos
6. En error → muestra toast via NotificationContext
```

React — `useState`: Declara una variable de estado reactiva. Cuando se llama al setter (`setSuppliers`), React vuelve a ejecutar el componente y actualiza el DOM con los nuevos valores. La UI es siempre un reflejo del estado actual.

React — `useEffect`: Ejecuta código con efectos secundarios (llamadas a APIs, suscripciones, timers) después de que React renderizó el componente. El array `[]` al final es la lista de dependencias: `[]` vacío significa "ejecutar solo al montar el componente", equivalente a `componentDidMount` en clases. Si se incluyen variables, se re-ejecuta cada vez que cambian.

Ejemplo (SuppliersList.jsx):

```
const [suppliers, setSuppliers] = useState([]);
const [loading, setLoading] = useState(true);

useEffect(() => {
  const load = async () => {
    try {
      const data = await supplierService.getAuthorizedSuppliers(user.id,
currentRole.role, currentRole.id_entity);
      setSuppliers(data);
```

```
    } catch {
      showError('Error al cargar proveedores');
    } finally {
      setLoading(false);
    }
  };
  load();
}, []); // [] = solo se ejecuta al montar el componente
```

Manejo de archivos (base64 ↔ Blob)

El backend envía archivos como **base64**:

```
Backend → base64 string
  → fileUtils.base64ToBlobUrl()
  → URL.createObjectURL(blob)
  → <img src={blobUrl} /> o <a href={blobUrl} download>
```

Para subir archivos:

```
<input type="file">
  → fileUtils.fileToBase64()
  → POST /elements/saveFile/:id  con { file: "base64...", fileName, mimeType }
```

---

5. Autenticación y roles

Flujo de login

```
1. /login → POST /auth/login
2. Backend responde con { token, user: { id, roles, ... } }
3. AuthContext guarda en localStorage: token, user, currentRole
4. authUtils.getAvailableRoles(user) mapea los roles disponibles
5. Si 1 rol → /dashboard
6. Si varios → /select-role para elegir → /dashboard
```

Mock login: credenciales vacías cargan datos de `data/mockUsers.js` (solo desarrollo).

Roles

Rol backend	Subtipo	Label UI	Acceso
`ADMIN`	—	Administrador	Total
`AUDITOR`	TECHNICAL	Auditor Técnico	Auditoría técnica, proveedores, recursos
`AUDITOR`	LEGAL	Auditor Legal	Documentos, inbox, vencimientos
`CUSTOMER` → `EMPRESA`	—	Empresa	Sus proveedores, recursos, configuración
`SUPPLIER` → `PROVEEDOR`	—	Proveedor	Sus propios datos, recursos, documentos

Forma del objeto user

```
user = {
  id: 1,
  firstName: "Nombre",
  lastName: "Apellido",
  username: "email@ejemplo.com",
  active: true,
  roles: ["SUPPLIER", "AUDITOR", "CUSTOMER", "ADMIN"],  // roles del backend
  suppliers: [
    { id_supplier: 1, company_name: "Empresa S.A.", cuit: "20-12345678-9", active: 1,
elements: [] }
  ],
  auditors: [
    { id_auditor: 2, registration_number: "12345678", type_auditor: "TECHNICAL" }
  ],
  clients: [
```

```
    { id_company_client: 1, rank: "boss", id_company: 1, companyDescription: "EDESAL"
}
  ]
}
```

Forma del objeto currentRole

Este objeto es lo que se guarda en localStorage y se usa en **casi todos los servicios**. Se construye en `RoleSelectionPage` al hacer `selectRole()`:

```
currentRole = {
  role: "EMPRESA",          // "ADMIN" | "AUDITOR" | "EMPRESA" | "PROVEEDOR"
  roleId: 1,
  type: "EMPRESA",          // Para AUDITOR: "TECHNICAL" o "LEGAL". Para otros: igual
a role.
  id_entity: 1,             // ID de la empresa, proveedor o auditor seleccionado
  entity_name: "EDESAL",    // Nombre legible de la entidad
  // + todas las propiedades del objeto entidad (id_company, rank, etc.)
}
```

> Los servicios que filtran por rol usan `currentRole.role`, `currentRole.id_entity` y el `user.id`. Si alguno de estos es `null` o `undefined`, las llamadas al backend pueden fallar o devolver datos incorrectos.

Timeout de sesión

`AuthContext` monitorea eventos `mousedown`, `keypress`, `scroll`, `touchstart`. Después de **60 minutos** sin actividad → logout automático.

Sincronización entre pestañas

`AuthContext` escucha el evento `storage` del browser. Si el usuario cierra sesión en otra pestaña, las demás detectan el cambio y redirigen a `/login` automáticamente.

Uso en cualquier componente

```
import { useAuth } from '../../context/AuthContext';

const { user, currentRole, isAdmin, isAuditorTecnico, isProveedor, logout } =
useAuth();
```

React — Context API: Permite compartir datos entre componentes sin pasarlos como props por cada nivel. Funciona en tres partes: `createContext()` crea el contenedor, `<Context.Provider value={...}>` lo envuelve en el árbol y provee los datos, y `useContext(Context)` los consume desde cualquier componente hijo. En este proyecto, `AuthProvider` envuelve toda la app en `main.jsx`, por eso `useAuth()` funciona en cualquier componente. El hook `useAuth()` es solo un atajo sobre `useContext(AuthContext)` que además lanza un error claro si se usa fuera del Provider.

---

6. Routing y navegación

`src/App.jsx` usa `createBrowserRouter` de React Router v7.

React Router DOM — conceptos clave:

- `createBrowserRouter`: define el árbol de rutas de la SPA. Cada ruta asocia una URL con un componente. Reemplaza a `<BrowserRouter>` + `<Routes>` del enfoque anterior.
- `<Outlet />`: marcador de posición dentro de un layout. Cuando una ruta hija está activa, React Router renderiza su componente en el lugar donde esté `<Outlet>`. Por eso `MainLayout` contiene `<Outlet>` y todas las páginas aparecen dentro del mismo frame con sidebar y navbar.
- `useNavigate()`: hook para navegar por código. Ej: `navigate('/dashboard')` redirige sin recargar la página.
- `useParams()`: hook para leer segmentos dinámicos de la URL. En `/proveedores/:id`, `useParams()` devuelve `{ id: "123" }`.
- `useLocation()`: hook para leer la URL actual completa, incluyendo `pathname`, `search` y `state` (datos pasados entre rutas sin mostrarlos en la URL).

```
/                           → redirect a /login
/login                      → LoginPage
/select-role                → RoleSelectionPage

(dentro de MainLayout)
/dashboard                  → DashboardHome
/proveedores                → SuppliersList
/proveedor                  → SupplierData
/proveedores/nuevo          → NewSupplier
/proveedores/:id            → SupplierDetail
/proveedores/:id/asociar-empresa → AssociateCompany
/auditores                  → AuditorsList
/auditores/tecnica          → TechnicalAudit
/auditores/tecnica/historial → AuditHistory
/auditores/:id              → AuditorDetail
/auditoria-legal            → LegalAuditDashboard
/auditoria-legal/inbox      → GlobalAuditInbox
/empresas                   → CompanyList
/empresas/nueva             → NewCompany
/empresas/editar/:id        → EditCompany
/configuracion              → CompanySettings
/recursos                   → ResourcesDashboard
/recursos/vehiculos         → VehiclesList
/recursos/vehiculos/nuevo   → NewVehicle
/recursos/empleados         → EmployeesList
/recursos/empleados/nuevo   → NewEmployee
/recursos/maquinaria        → MachineryList
/recursos/maquinaria/nueva  → NewMachinery
/recursos/configurar-documentacion → DocumentationConfig
/recursos/documentacion/:type/:id  → ResourceDocumentationView
```

```
/usuarios                    → UsersList
/usuarios/:id                → UserDetail
/documentos/:status?         → ProviderDocuments
/404  /500  *                → NotFound / ServerError
```

---

7. Layout y componentes de navegación

MainLayout.jsx

Wrapper de toda la app autenticada. Contiene sidebar + navbar + `<Outlet>` (página activa). El sidebar puede estar **pinado** (siempre visible) o **flotante** (se expande al hover). En mobile hay toggle.

> **React — props `children`:** En React, `children` es la prop especial que representa el contenido que se pasa entre las etiquetas de apertura y cierre de un componente. `<Outlet>` en este layout cumple el mismo rol: es donde React Router inyecta el componente de la página activa. Cuando el usuario navega a `/proveedores`, React Router renderiza `SuppliersList` dentro del `<Outlet>` de `MainLayout`, manteniendo el sidebar y navbar intactos.

Navbar.jsx

- Breadcrumbs dinámicos por URL
- Selector de rol (cambia el rol sin logout)
- Menú de usuario con logout
- El color del navbar **cambia según el rol activo**

Sidebar.jsx

Menú dinámico según rol:

Rol	Ítems
PROVEEDOR	Dashboard, Mis Datos, Recursos, Documentos
AUDITOR	Dashboard, Auditoría (expandible), Proveedores, Recursos, Documentos
EMPRESA	Dashboard, Proveedores, Recursos, Documentos, Configuración
ADMIN	Dashboard, Usuarios, Empresas, Auditores, Proveedores, Mis Datos, Recursos, Auditorías, Documentos

Soporta submenús expandibles, pin/unpin, y swipe-to-close en mobile.

8. Componentes principales y cómo se usan

React — Componentes y props: Un componente en React es una función que recibe `props` (propiedades) y retorna JSX (HTML-like syntax). Las props son la forma de pasar datos de un componente padre a uno hijo. Son de solo lectura: el hijo nunca modifica las props recibidas. En los ejemplos a continuación, todo lo que se pasa entre `< >` son props (`value`, `header`, `title`, `icon`, `onClick`, etc.).

AppTable.jsx

Wrapper sobre `DataTable` de PrimeReact. Abstrae paginación, ordenamiento y soporte mobile.

```
<AppTable value={suppliers} header="Proveedores" emptyMessage="Sin resultados">
  <Column field="businessName" header="Razón Social" sortable />
  <Column body={actionsTemplate} header="Acciones" />
</AppTable>
```

StatCard.jsx

Tarjeta KPI para dashboards. Acepta `children` para mostrar filas de detalle debajo del valor principal. Es clickable y navega a la sección correspondiente.

```
<StatCard title="Proveedores" value={42} icon="pi-briefcase" type="primary" onClick=
{() => navigate('/proveedores')}>
  <DetailRow label="Activos" value={38} />
  <DetailRow label="Inactivos" value={4} />
</StatCard>
```

Tipos de color disponibles: `primary` (indigo), `success` (lime), `warning` (amber), `danger` (red), `info` (sky).

PageHeader.jsx

Encabezado estándar de página con título, subtítulo y slot para acciones.

```
<PageHeader title="Proveedores" subtitle="Gestión de contratistas" actions=
{<PrimaryButton label="Nuevo" />} />
```

PrimaryButton.jsx

Botón indigo con soporte de `mobileLabel` (texto diferente en pantallas pequeñas).

```
<PrimaryButton label="Guardar cambios" mobileLabel="Guardar" icon="pi pi-save"
onClick={handleSave} />
```

Input.jsx y Select.jsx

Campos de texto y dropdown con estilos del proyecto. Soportan `size="sm"` / `size="md"`, ícono izquierdo, y estado deshabilitado.

WizardSteps.jsx

Barra de progreso para formularios multi-paso.

```
<WizardSteps steps={['General', 'Ubicación', 'Contactos', 'Documentos']} currentStep=
{2} completedSteps={[0, 1]} />
```

DocumentCard.jsx

Tarjeta de documento con estado semántico y acciones según rol. Los estados posibles son:

Estado	Color	Significado
`VIGENTE`	Verde	Documento aprobado y dentro de vigencia
`EN_REVISION`	Azul	Subido, pendiente de aprobación del auditor
`PENDIENTE`	Gris	No fue subido aún
`VENCIDO`	Rojo	Fecha de vencimiento superada
`CON_OBSERVACION`	Amarillo	Observado por el auditor, requiere corrección

Muestra la fecha de vencimiento y emite advertencia visual si vence en los próximos 10 días.

Badges (Badges.jsx)

```
<StatusBadge status="VIGENTE" />   // chip verde
<RiskBadge level="ALTO" />         // chip rojo
<BooleanBadge value={true} />      // "Sí" verde / "No" rojo
```

ConfirmationModal.jsx

Modal de confirmación antes de acciones destructivas (eliminar, desasociar).

```
<ConfirmationModal visible={show} onConfirm={handleDelete} title="¿Eliminar?"
severity="danger" />
```

ObservationModal.jsx / AuditDocumentModal.jsx

Modales del flujo de auditoría. El auditor los usa para agregar observaciones a documentos o aprobar/rechazar archivos. Reciben el documento como prop y llaman a `auditorService.saveFileAudit()`.

MultiSelect.jsx

Wrapper sobre el MultiSelect de PrimeReact con checkboxes indigo del proyecto.

```
<MultiSelect label="Empresas" value={selected} onChange={setSelected} options=
{companies} optionLabel="name" />
```

LoadingOverlay.jsx

Spinner de carga superpuesto al contenido mientras se fetcha del backend.

DocumentEntityTable.jsx

Tabla específica para documentos agrupados por entidad (proveedor o recurso). Tiene columnas de estado, vencimiento, y acciones de auditoría.

AdminSupplierFilterModal.jsx

Modal de filtros avanzados para la lista de proveedores. Solo visible para admins y empresas. Permite filtrar por grupo, empresa, estado y riesgo.

Formularios de recursos (components/resources/forms/)

Componente	Recurso	Campos principales	Se usa en
`EmployeeData.jsx`	Empleado	Nombre, DNI, legajo, puesto, área	`NewEmployee.jsx`
`VehicleData.jsx`	Vehículo	Patente, marca, modelo, año, combustible	`NewVehicle.jsx`
`MachineryData.jsx`	Maquinaria	Código, nombre, marca, serie, tipo	`NewMachinery.jsx`
`DocumentsData.jsx`	Documentos	Lista de docs requeridos con estado	Wizard proveedor
`AssignmentData.jsx`	Asignación	A qué empresa/proyecto está asignado	Wizard proveedor
`SupplierStep*.jsx`	Pasos del wizard	Ver sección Alta de proveedor	`NewSupplier.jsx`

9. Páginas y flujos

Login y selección de rol

```
/login → POST /auth/login → GET /auth/users/:id (perfil completo)
  → 1 solo rol disponible → AuthContext.selectRole() → /dashboard
  → Múltiples roles → /select-role
      → RoleSelectionPage muestra una RoleCard por cada rol
      → Si el rol tiene múltiples entidades (ej: PROVEEDOR con 2 empresas) → Dropdown
      → Al confirmar: AuthContext.selectRole({ role, type, id_entity, entity_name })
      → /dashboard
```

`RoleSelectionPage` mapea los datos del usuario a tarjetas con `authUtils.getAvailableRoles(user)`. Cada `RoleCard` puede tener un `Dropdown` interno si el rol tiene más de una entidad asociada (por ejemplo, un usuario que es proveedor de dos empresas distintas). Para AUDITOR, el `type` (`TECHNICAL` o `LEGAL`) se extrae de la entidad seleccionada.

Alta de proveedor (Wizard 5 pasos)

Manejado por `useSupplierForm.js`:

```
Paso 1: SupplierStepGeneral       (CUIT, razón social, rubro)
Paso 2: SupplierStepGroupCompany  (grupo y empresa cliente)
Paso 3: SupplierStepLocation      (país, provincia, ciudad)
Paso 4: SupplierStepContacts      (contactos: nombre, email, tel)
Paso 5: SupplierStepDocuments     (docs requeridos por el grupo)
→ POST /supplier/save
```

Dashboard

El dashboard adapta sus KPIs según el rol activo:

Rol	Cards mostradas
ADMIN	Empresas, Proveedores (Habilitados/No Habilitados), Recursos (emp+veh+maq), Auditoría pendiente
EMPRESA	Proveedores (Activos/Doc. pend./Suspendidos), Empleados, Vehículos, Maquinaria
PROVEEDOR	Mi Legajo, Empleados, Vehículos, Maquinaria
AUDITOR TÉCNICO	Total asignados (con desglose por grupo), Aprobados, Pendientes, Rechazados
AUDITOR LEGAL	Docs en revisión, Proveedores, Vencidos/Por vencer, Observados

Cada card es clickable y navega a la sección correspondiente. Los datos provienen de tres fuentes en paralelo:

- `dashboardService.getStats()` → datos del backend
- `useResourceStats()` → estadísticas computadas localmente desde los hooks de recursos
- `supplierService.getAuthorizedSuppliers()` + `groupService.getAll()` → para calcular el desglose de proveedores por grupo (especialmente útil para AUDITOR TÉCNICO)

Lógica de estado de proveedores (campo `active`):

- `active === 0` → `HABILITADO`
- `active === 1` → `NO HABILITADO`
- `active === 2` → `SUSPENDIDO`

Los gráficos (Bar stacked y Line chart) tienen datos estáticos. Las alertas también son estáticas.

Auditoría técnica

```
/auditores/tecnica → auditorService.getPendingFiles()
  → Lista de documentos pendientes
  → Click → AuditDocumentModal
  → POST /audit/save { fileId, status, observation }
```

Documentación de recursos

```
/recursos/empleados/:id  (o /vehiculos, /maquinaria)
  → elementService.getById(id)
  → groupService.getByElement(id)
  → groupService.specificResource(idSupplier, idGroup, idActive, id)
  → Lista de docs requeridos para ese recurso
  → Subida: POST /elements/saveFile/:id
```

ResourcesDashboard (/recursos)

Panel de administración de todos los recursos. Muestra 3 `StatCards` (Empleados, Vehículos, Maquinaria) con métricas de habilitación usando `useResourceStats`. Debajo, un `TabView` de PrimeReact embebe directamente las listas (`EmployeesList`, `VehiclesList`, `MachineryList`) con `isEmbedded={true}`.

Comportamiento especial para ADMIN: Al entrar, abre automáticamente el `AdminSupplierFilterModal` para que seleccione grupo → empresa → proveedor. Sin esa selección, no se muestran los tabs. El proveedor seleccionado se pasa como `explicitIdSupplier` a `useResourceStats` y a las listas embebidas.

ProviderDocuments (/documentos/:status?)

Vista de documentación agrupada por tipo de entidad (Legajo, Empleados, Vehículos, Maquinaria). El parámetro `status` de la URL actúa como filtro preseleccionado:

URL param	Filtro aplicado
`general`	Todos los documentos
`pendientes`	Sin subir + Vencidos (estado `VENCIDO`)
`por-vencer`	Vencimiento en los próximos 10 días (solo docs aún vigentes)
`observados`	Con observaciones
`en-revision`	Bajo auditoría
`vigentes`	Aprobados y vigentes

> **Nota:** Los documentos con estado `VENCIDO` aparecen únicamente en `pendientes`, no en `por-vencer`. El filtro `por-vencer` muestra solo los que tienen `isExpiringSoon = true` (vencimiento ≤10 días pero aún no vencidos). El sidebar los agrupa bajo el label **"Pendientes / Vencidos"**.

Cada tab contiene un `DocumentEntityTable` con el filtro activo. Las `StatCards` muestran barras de progreso de carga y cumplimiento calculadas en tiempo real por el propio `DocumentEntityTable` via callback `onStatsChange`.

10. Servicios (capa API)

Todos los servicios son módulos ES con funciones `async` que usan la instancia `api` de `axiosConfig.js`.

elementService.js

Exportado como objeto `default`. Las funciones de mapeo reciben `currentRole` y `suppliers` para resolver el nombre del proveedor según el rol activo.

```
import elementService from '../services/elementService';

const elementos = await elementService.getBySupplier(idSupplier);
const mapeado = elementService.mapToUIVehicle(e, idSupplier, currentRole, suppliers);
```

Campos en los objetos mapeados:

- `docStatus` — estado documental del recurso proveniente del backend (VIGENTE, VENCIDO, SIN_REQUISITOS, PENDIENTE, etc.). Por defecto `SIN_REQUISITOS`.
- `estado` — estado efectivo del recurso, calculado por `computeResourceEstado(rawEstado, docStatus)`:
  - Si `rawEstado === 'DADO DE BAJA'` → `'DADO DE BAJA'`
  - Si `docStatus` es `HABILITADO` o `SIN_REQUISITOS` → `'HABILITADO'`
  - Cualquier otro `docStatus` → `'NO HABILITADO'`
- `expirationDate` — fecha de vencimiento del documento
- `motivo` — motivo de observación o rechazo
- `proveedor` — nombre resuelto según el rol (si es PROVEEDOR usa `currentRole.entity_name`, si no busca en el objeto del backend)

supplierService.js

- `getAll()`, `getAuthorizedSuppliers(userId, role, entityId)`, `getByCompany(id)`
- `getById(cuit)` — datos básicos, `getWithDocuments(cuit)` — datos + documentos (más lento)
- `create(data)`, `update(data)`, `delete(id)`
- `getAssociations(cuit)`, `associateCompanies(data)`
- `mapToUISupplier(raw)` → normaliza el objeto backend al formato UI

Mapeo del campo `active` en `mapToUISupplier`:

- `active === 0` → `estado: 'HABILITADO'`
- `active === 1` → `estado: 'NO HABILITADO'`
- `active === 2` → `estado: 'SUSPENDIDO'`

groupService.js

Usa deduplicación de requests internamente (Map `_inFlight`) para evitar llamadas paralelas duplicadas al mismo endpoint.

fileService.js

- `getFile(id)` → retorna `Blob`
- `uploadFileForElement(elementId, fileData)`
- `updateDateForElement(elementId, dateData)`
- `deleteFile(id)`
- `formatForBackend(date)` → convierte fecha a ISO para el backend

activeService.js

Servicio mínimo con una sola función. Obtiene los tipos de activos disponibles por tipo:

- `getByType(idActiveType)` → GET `/actives/getByType/:id`

Se usa internamente en `useSupplierForm` para cargar el catálogo de activos del legajo (tipo 5) cuando el wizard está en modo admin o creación.

requirementService.js

Maneja el catálogo de documentos requeridos y su asignación a grupos:

- `getListRequirements({ idGroup, idActiveType })` → GET `/list_requirements?idGroup=&idActiveType=`
- `getGroupRequirementsDetails({ idSupplier, idGroup, idActiveType })` → GET `/group_requirements/details`
- `getSupplierDocuments(cuit)` → GET `/supplier/oneSupplierForDocument?cuit=`
- `saveGroupRequirement(payload)` → POST `/group_requirements/save`

> Nota: este servicio tiene `console.log` de debug activos en `getListRequirements`. Conviene limpiarlos antes de producción.

companyClientService.js

Maneja el vínculo entre usuarios y empresas (tabla `company_client`):

- `getAll()` → GET `/company_client/getAll`
- `create(companyClientDTO)` → POST `/company_client/save`

auditorService.js

- `getAll()`, `getById(id)`, `create(data)`

- `assignCompany(auditorId, companyId)`, `removeCompany(auditorId, companyId)`
- `getPendingFiles()` → archivos pendientes de auditoría
- `saveFileAudit(data)` → guarda resultado de auditoría (`{ fileId, status, observation }`)

userService.js

- `getAll()`, `getById(id)`, `create(data)`, `update(id, data)`, `delete(id)`
- `assignRole(userId, roles)`
- `getAvailableRoles()` → lista estática (mock)

companyService.js

- `getAll()`, `getByGroup(groupId)`, `create(data)`
- `updateStatus(companyId, requiredTechnical)` → activa/desactiva requisito de auditoría técnica

dashboardService.js

- `getStats(userId, role, entityId)` → estadísticas generales del backend según rol

---

11. Hooks personalizados

React — Hooks personalizados (Custom Hooks): Un hook personalizado es simplemente una función cuyo nombre empieza con `use` y que puede llamar a otros hooks de React (`useState`, `useEffect`, etc.). Sirven para extraer lógica reutilizable fuera de los componentes. La regla es que solo pueden llamarse desde componentes funcionales u otros hooks — nunca desde funciones normales, condicionales o loops. En este proyecto, todos los archivos en `hooks/` son custom hooks que encapsulan fetching, transformación de datos y efectos secundarios para que las páginas sean más simples.

useResourceStats.js

Compone datos de `useEmployees`, `useVehicles` y `useMachinery`, y calcula estadísticas localmente con `useMemo`. No hace ninguna llamada HTTP propia.

```
const { stats, loading, suppliersWithReview, totalProviders } =
useResourceStats(explicitIdSupplier);
```

Estructura de `stats`:

```
stats = {
  employees: { total, habilitados, enRevision, conObservacion, vencidos, docPendiente,
expiringSoon, pct, providerCount },
  vehicles:  { ... mismos campos ... },
  machinery: { ... mismos campos ... }
}
```

React — `useMemo`: Memoriza el resultado de un cálculo costoso. Solo lo recalcula cuando cambian sus dependencias (el array al final). Acá se usa para que las estadísticas de empleados, vehículos y maquinaria no se recalculen en cada render del componente, sino solo cuando los datos reales cambian.

React — `useCallback`: Similar a `useMemo` pero para funciones. Memoriza la referencia de una función para que no se cree una nueva instancia en cada render. Se usa en `useSupplierForm` para handlers como `handleChange`, `handleFileUpload`, etc., evitando re-renders innecesarios en componentes hijos que reciben esas funciones como props.

Estados que considera habilitado: `COMPLETA`, `VIGENTE`, `APROBADO`, `SIN_REQUISITOS`, `HABILITADO`

Estados que considera pendiente: `PENDIENTE`, `INCOMPLETA`, `FALTANTE`, `VENCIDO`, `CON_OBSERVACION`, `OBSERVADO`, `RECHAZADO`, `NO HABILITADO`, `NO_HABILITADO`

`expiringSoon`: recursos cuya fecha de vencimiento está entre hoy y los próximos 10 días (excluyendo los ya VENCIDO).

`explicitIdSupplier`: parámetro opcional para filtrar stats de un proveedor específico. Si es `null`, trae todos los autorizados según el rol activo.

useSupplier.js

Gestiona datos del proveedor + documentos:

```
const { supplier, documents, loading, error, updateSupplier, refresh } =
useSupplier(cuit);
```

Calcula el estado de cada documento y detecta vencimientos próximos (10 días).

useSupplierForm.js

Maneja el estado completo del wizard. Es el hook más complejo del proyecto — centraliza todo el estado de un formulario de 5 pasos para que el componente `SupplierForm.jsx` solo se ocupe del renderizado. Recibe un objeto de configuración:

```
const form = useSupplierForm({
  initialData,          // datos iniciales del proveedor (para edición)
  isWizardMode,         // true = creación, false = edición de un paso
  readOnly,             // bloquea cambios
  isAdmin,              // habilita el paso "Grupo y Empresa"
  isAuditor,
  groups,               // lista de grupos disponibles
  availableCompanies,   // empresas disponibles para asignar
  availableRequirements,// catálogo de documentos disponibles
  onSubmit              // callback al guardar
});
```

Pasos del wizard según rol:

- ADMIN / AUDITOR: 5 pasos — Proveedor, Grupo y Empresa, Ubicación, Contactos, Documentos
- Resto: 4 pasos — se omite "Grupo y Empresa"

Dirty steps: cada vez que el usuario modifica un campo se llama `markStepDirty(stepIndex)`. Al navegar entre pasos sin guardar, la UI puede mostrar `UnsavedChangesModal` si hay pasos sucios.

Comportamiento crítico al subir un archivo:

1. El archivo se convierte a `blob:` URL local para previsualización inmediata
2. Se setea `estado: 'EN REVISIÓN'` en el documento
3. Si el documento no tenía `fechaVencimiento`, se calcula automáticamente según `frecuencia`:
   - MENSUAL → +1 mes, TRIMESTRAL → +3 meses, SEMESTRAL → +6 meses, ANUAL → +1 año
4. Al guardar (`handleSubmit`), solo se envían al backend los documentos con `modified: true`

Comportamiento al eliminar un archivo auditado:

- Si el documento ya tiene auditorías (`hasAudits: true`) y no fue modificado localmente → eliminación bloqueada
- Si fue modificado localmente → revierte al archivo original y al estado anterior (`oldArchivo`, `oldEstado`)

Visualización de archivos (`handleViewFile`):

1. Si es un `blob:` URL local → `window.open` directo
2. Si tiene `id_file_submitted` → `fileService.getFile(id)` → crea nuevo blob URL → `window.open`
3. Si falla → intenta con `fileUrl` como fallback

useEmployees.js / useVehicles.js / useMachinery.js

Misma estructura. Exponen: `employees/vehicles/machinery`, `loading`, `create`, `update`, `remove`.

Internamente usan `elementService.getAuthorized()` o `elementService.getBySupplier()` según el rol.

usePermissions.js

Centraliza permisos por rol:

```
const { canEdit, canDelete, canAudit, canViewAll } = usePermissions();
```

---

12. Gestión de estado

Estado global (React Context)

Context	Qué almacena
`AuthContext`	`user`, `currentRole`, `token`, flags de rol (`isAdmin`, etc.)
`NotificationContext`	ref al toast, `showSuccess()`, `showError()`, `showInfo()`, `showWarn()`
`BreadcrumbContext`	`labels` — map de `{ path: label }` para sobreescribir breadcrumbs dinámicos

`BreadcrumbContext` — Permite a páginas de detalle (ej: `/auditores/:id`) inyectar un label legible (nombre del auditor) en lugar del ID crudo que mostraría `Navbar.jsx`. Se usa con `setLabel(path, label)` al montar y `clearLabel(path)` al desmontar.

```
import { useBreadcrumb } from '../../context/BreadcrumbContext';
const { setLabel, clearLabel } = useBreadcrumb();
```

No hay Redux ni Zustand. El estado se maneja con Context + hooks locales.

> **React — cuándo usar Context vs estado local:** El estado local (`useState` dentro de un componente) es suficiente para datos que solo usa ese componente o sus hijos directos. El Context se usa cuando muchos componentes en distintos niveles del árbol necesitan el mismo dato (como el usuario autenticado o el sistema de notificaciones). Agregar Context innecesariamente complica el código; la regla es empezar con estado local y elevar solo cuando sea necesario.

Persistencia en localStorage

`AuthContext` persiste:

- `token` — JWT del backend
- `user` — objeto completo del usuario
- `currentRole` — rol activo
- `currentEntity` — entidad asociada al rol

Se leen sincrónicamente al montar para evitar flash de login en recarga.

Estado local por componente

La mayoría de páginas manejan su propio estado con `useState`:

- `loading: boolean`
- `data: array | object`
- `error: string | null`
- `selectedItem: object | null`
- `showModal: boolean`

---

13. Detalles de implementación relevantes

Deduplicación de requests en groupService

El servicio usa un Map `_inFlight` internamente. Si se lanza el mismo request dos veces en paralelo antes de que el primero resuelva, ambos reciben la misma promesa. Evita llamadas duplicadas en componentes que montan simultáneamente.

Locale en español (PrimeReact)

`main.jsx` configura el locale de PrimeReact en español. Afecta calendarios, datepickers y cualquier componente con texto de UI (días, meses, "hoy", etc.).

Manejo de fechas y zonas horarias

El backend envía fechas como strings `YYYY-MM-DD`. El frontend las parsea fijando la hora a las 12:00 local para evitar que el timezone desplace el día:

```
const d = new Date(+year, +month - 1, +day);
d.setHours(0, 0, 0, 0);
```

`fileService.formatForBackend(date)` hace el proceso inverso al enviar.

Monitoreo de inactividad

`AuthContext` registra los eventos `mousedown`, `keypress`, `scroll` y `touchstart` en `document`. Cada evento reinicia un timer de 60 minutos. Al expirar, llama a `logout()` automáticamente.

Locale y configuración regional

El CUIT argentino se formatea con `formatUtils.formatCUIT()`:

```
formatCUIT('20123456789') // → '20-12345678-9'
```

---

14. Sistema de diseño

Paleta "Clean Industrial" documentada en `DESIGN.md`:

Token	Color	Hex	Uso
`primary`	Indigo	`#6366f1`	Botones, links, focus ring
`primary-hover`	Indigo dark	`#4f46e5`	Hover
`secondary`	Slate	`#64748b`	Texto secundario
`success`	Lime	`#84cc16`	OK, vigente, habilitado
`danger`	Red	`#ef4444`	Error, vencido, rechazado
`warning`	Amber	`#f59e0b`	Por vencer, pendiente
`info`	Sky	`#0ea5e9`	En revisión, información

`index.css` sobrescribe estilos de PrimeReact para que sigan el design system: calendarios compactos, checkboxes indigo, toasts con borde izquierdo de color, DataTable con hover suave.

---

15. Limitaciones y deuda técnica conocida

Área	Descripción
Token JWT	Se guarda en `localStorage` pero **no se envía automáticamente** en los headers de Axios. Si el backend empieza a requerir autenticación en sus endpoints, hay que agregar un interceptor en `axiosConfig.js`: `api.interceptors.request.use(config => { config.headers.Authorization = \` Bearer ${token}`; return config; })`
URL del backend hardcodeada	`http://localhost:8080` está fija en `axiosConfig.js`. No usa variables de entorno.
Gráficos del dashboard estáticos	El Bar chart ("Estatus por Recursos") y el Line chart ("Cumplimiento Global") tienen datos hardcodeados. No consumen endpoints reales.
Alertas del dashboard estáticas	La tabla "Alertas Recientes" tiene 3 filas hardcodeadas. No hay endpoint de alertas integrado.
`console.log` en producción	`requirementService.js` tiene `console.log` de debug activos en `getListRequirements`.
Enlace "Solicitar vinculación"	En `RoleSelectionPage` hay un `<a href="#">` sin destino real.
Autorización de documentación pendiente por empresa	Falta implementar que la empresa pueda autorizar/exceptuar documentos pendientes de un proveedor, de modo que esos documentos no cuenten negativamente en el semáforo del proveedor hasta que sean presentados.
Badges de cantidades y alertas en el sidebar	Los badges que muestran contadores (documentos pendientes, alertas) en el sidebar están maquetados pero no consumen datos reales. Desmaquetarlos y conectarlos a los endpoints correspondientes.
Auditoría técnica en semáforo del proveedor	El estado final del proveedor y su semáforo no considera si tiene auditoría técnica pendiente o rechazada. Hay que incorporar el resultado de la auditoría técnica (`TechnicalAudit`) al cálculo del estado del proveedor para los proveedores que la requieren.

16. Cómo correr el proyecto

Requisitos

- Node.js >= 18
- Backend corriendo en `http://localhost:8080`

Comandos

```
npm install          # Instalar dependencias
npm run dev          # Dev server (puerto 5173 por defecto)
npm run build        # Build de producción
npm run preview      # Preview del build
npm run lint         # ESLint
```

Login de desarrollo

Con credenciales vacías se cargan datos mock. Permite probar todos los roles sin backend activo.

Cambiar URL del backend

Actualmente hardcodeada en `src/api/axiosConfig.js`. Se recomienda migrar a variable de entorno:

```
// src/api/axiosConfig.js
const api = axios.create({ baseURL: import.meta.env.VITE_API_URL ||
'http://localhost:8080' });
```

Y crear un `.env.local`:

```
VITE_API_URL=http://localhost:8080
```

17. Usuarios de prueba

Todos los usuarios tienen contraseña `123`.

Usuario	Rol	Subtipo	Acceso en UI
`lmorandini`	ADMIN	—	Administrador total
`bpaez@gmail.com`	ADMIN	—	Administrador total
`bpaez@gmail.com`	SUPPLIER	—	Proveedor (mismo usuario, doble rol)
`rchaves`	AUDITOR	LEGAL	Auditor Legal
`mmauro`	AUDITOR	TECHNICAL	Auditor Técnico
`pfalcon`	CUSTOMER	—	Empresa

> `bpaez@gmail.com` tiene dos roles activos (ADMIN y SUPPLIER). Al loguearse, se le presenta la pantalla de selección de rol.

Última actualización: 2026-03-31.